

Lesson 4: State & Interactivity

FlutterLearn | Module 1: Flutter Foundations

Why stateless isn't enough

A StatelessWidget is frozen after it's built. But real apps change: counters count, text fields fill, toggles flip. For UI that changes over time, Flutter gives you StatefulWidget.

The trick: a stateful widget is split in two. The widget itself is still immutable and lightweight — it gets thrown away and recreated freely. The mutable data lives in a separate State object that Flutter keeps alive.

The pattern

```

class Counter extends StatefulWidget {
  const Counter({super.key});

  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int count = 0; // mutable state lives here

  void _increment() {
    setState(() {
      count++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text('Count: $count', style: const TextStyle(fontSize: 24)),
        ElevatedButton(
          onPressed: _increment,
          child: const Text('Increment'),
        ),
      ],
    );
  }
}

```

Three things to notice:

1. `createState()` returns the `State` object where your variables live.
2. `setState()` is the magic call. You change your variables *inside* it, and it tells Flutter: "my data changed, rebuild me." Flutter then calls `build()` again with the new values.
3. `onPressed: _increment` — note there are no parentheses. You're passing the function itself as the callback, not calling it.

The golden rule

Changing a variable without `setState()` changes the data but not the screen. Flutter has no idea anything happened, so `build()` never re-runs and the UI stays stale. This single mistake causes

half of all beginner "my button does nothing" bugs.

```
// WRONG: count changes, screen doesn't
void _increment() {
  count++;
}

// RIGHT: screen rebuilds with the new count
void _increment() {
  setState(() {
    count++;
  });
}
```

Where you've seen this before

Think back: StatelessWidget describes UI that never changes. StatefulWidget + setState() is how UI *becomes* changeable. Between Lesson 2's static widgets, Lesson 3's layouts, and this lesson's state, you now have everything needed to build genuinely interactive apps.

Key terms

- State — data that can change over the lifetime of a widget.
- setState() — notifies Flutter that state changed; triggers a rebuild.
- State object — the long-lived companion of a StatefulWidget that holds mutable data.